

Révisions

👉 [Fiche.pdf](#) – [Télécharger la fiche de révisions](#)

Ce qu'il faut savoir

- connaître l'algorithme du **tri par insertion**

```
# Tri par insertion : on insère chaque élément à sa place
# dans la partie déjà triée du tableau
def tri_insertion(t):
    # On commence au deuxième élément
    for i in range(1, len(t)):
        x = t[i]          # Élément à insérer
        j = i - 1         # Indice de départ dans la partie triée
        # On décale les éléments plus grands que x
        while j >= 0 and t[j] > x:
            t[j + 1] = t[j]
            j -= 1
        # On insère x à la bonne position
        t[j + 1] = x
```

- connaître l'algorithme du **tri par sélection**

```
# Tri par sélection : on cherche le minimum
# et on le place à sa position définitive
def tri_selection(t):
    # Parcours du tableau sauf le dernier élément
    for i in range(len(t) - 1):
        j_min = i        # Indice du minimum
        # Recherche du minimum dans la partie non triée
        for j in range(i + 1, len(t)):
            if t[j] < t[j_min]:
                j_min = j
        # Échange si nécessaire
        if j_min != i:
            t[i], t[j_min] = t[j_min], t[i]
```

- savoir que l'algorithme du tri par insertion et l'algorithme du tri par sélection ont tous deux une **complexité en temps dans le pire des cas en $O(n^2)$** (quadratique)

Ce qu'il faut savoir faire

- être capable d'analyser et d'expliquer (faire tourner « à la main ») les algorithmes de tri par insertion et par sélection sur un exemple donné
- être capable d'implémenter en Python les algorithmes de tri par insertion et par sélection

Approfondissements

Tri par insertion en C

```
// Tri par insertion : insertion progressive des éléments
// dans la partie déjà triée du tableau
void tri_insertion(int t[], int n) {
    // On commence au deuxième élément
    for (int i = 1; i < n; i++) {
        int x = t[i];          // Élément à insérer
        int j = i - 1;        // Indice dans la partie triée
        // Décalage des éléments plus grands que x
        while (j >= 0 && t[j] > x) {
            t[j + 1] = t[j];
            j--;
        }
        // Insertion de x à la bonne place
        t[j + 1] = x;
    }
}
```

Tri par insertion en OCaml

```
(* Tri par insertion : on insère chaque élément
   à sa place dans la partie déjà triée *)
let tri_insertion (t : int array) : unit =
    let n = Array.length t in
    (* On commence au deuxième élément *)
    for i = 1 to n - 1 do
        let x = t.(i) in          (* Élément à insérer *)
        let j = ref (i - 1) in    (* Indice dans la partie triée *)
        (* Décalage des éléments plus grands que x *)
        while !j >= 0 && t.(!j) > x do
            t.(!j + 1) <- t.(!j);
            j := !j - 1
        done;
        (* Insertion de x à la bonne place *)
        t.(!j + 1) <- x
    done
```

Tri par sélection en C

```
// Tri par sélection : on place successivement
// le plus petit élément à sa position définitive
void tri_selection(int t[], int n) {
    // Parcours du tableau sauf le dernier élément
    for (int i = 0; i < n - 1; i++) {
        int j_min = i; // Indice du minimum
        // Recherche du minimum dans la partie non triée
        for (int j = i + 1; j < n; j++) {
            if (t[j] < t[j_min]) {
                j_min = j;
            }
        }
        // Échange si le minimum n'est pas déjà en place
        if (j_min != i) {
            int tmp = t[i];
            t[i] = t[j_min];
            t[j_min] = tmp;
        }
    }
}
```

Tri par sélection en OCaml

```
(* Tri par sélection : on cherche le minimum
   et on le place à sa position définitive *)
let tri_selection (t : int array) : unit =
    let n = Array.length t in
    (* Parcours du tableau sauf le dernier élément *)
    for i = 0 to n - 2 do
        let j_min = ref i in (* Indice du minimum *)
        (* Recherche du minimum dans la partie non triée *)
        for j = i + 1 to n - 1 do
            if t.(j) < t.(!j_min) then j_min := j
        done;
        (* Échange si nécessaire *)
        if !j_min <> i then (
            let tmp = t.(i) in
            t.(i) <- t.(!j_min);
            t.(!j_min) <- tmp
        )
    done
```